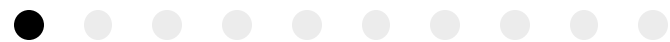


Learning assistant

- TRIKI Yasmine
- MALGACHE Estelle
- NZEGAING Christien

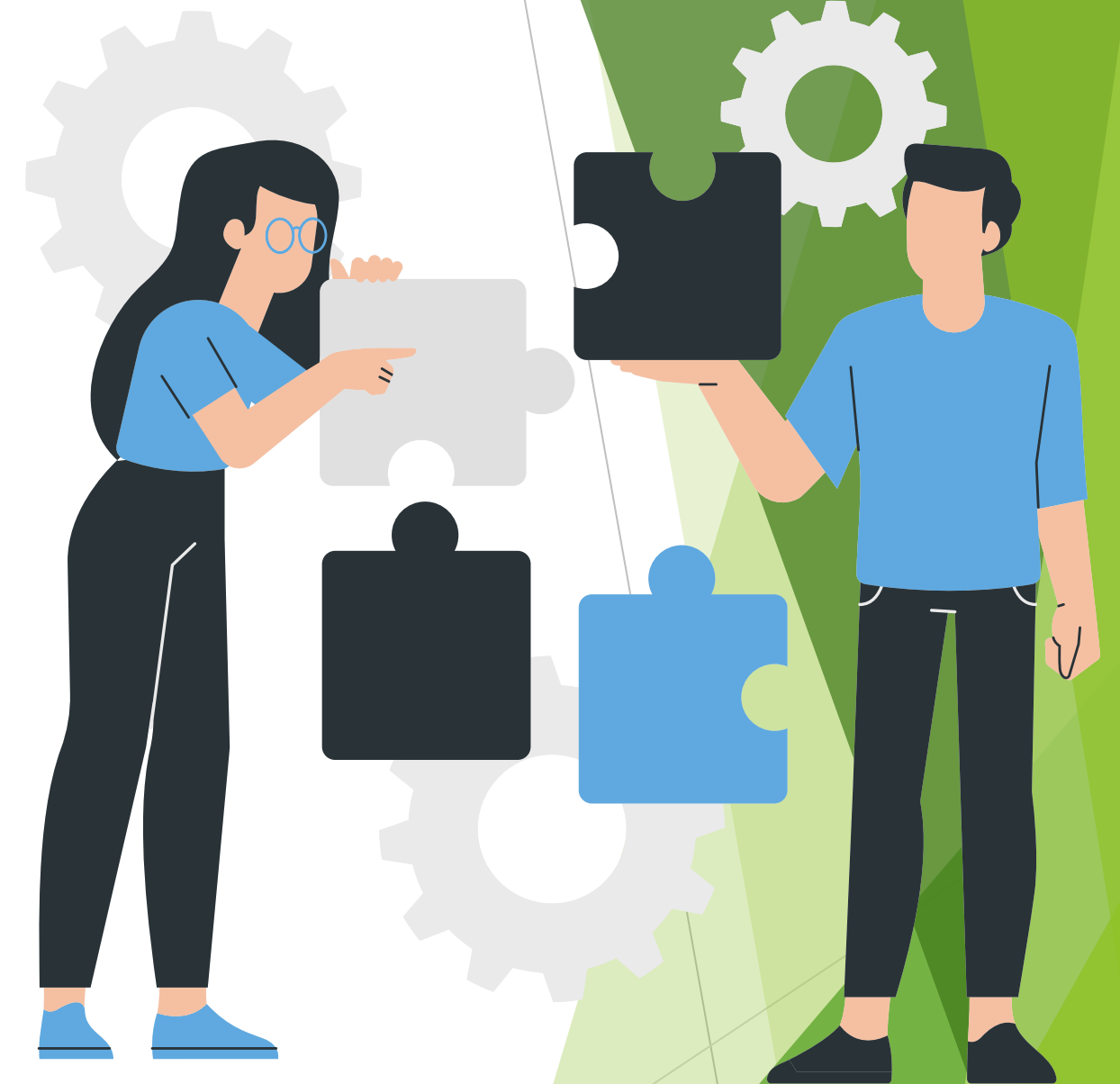
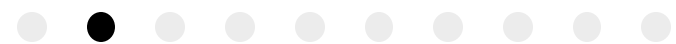


PLAN

- ▶ Problématique & objectif
- ▶ La cible
- ▶ Cas d'usage
- ▶ Fonctionnalités
- ▶ Maquette UX
- ▶ Architecture globale
- ▶ Outils et plateformes technologiques
- ▶ Pipeline & Dataset
- ▶ Architecture des modèles
- ▶ Cloud
- ▶ Perspectives futures & Feedback

Problématique & Objectif

- Les contenus pédagogiques sont souvent difficile à assimiler.
- Les apprenants perdent du temps à identifier l'essentiel et à créer des supports de révision efficace

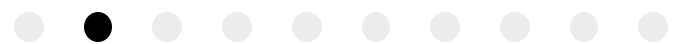


Problématique & Objectif

2

Concevoir un assistant intelligent capable de :

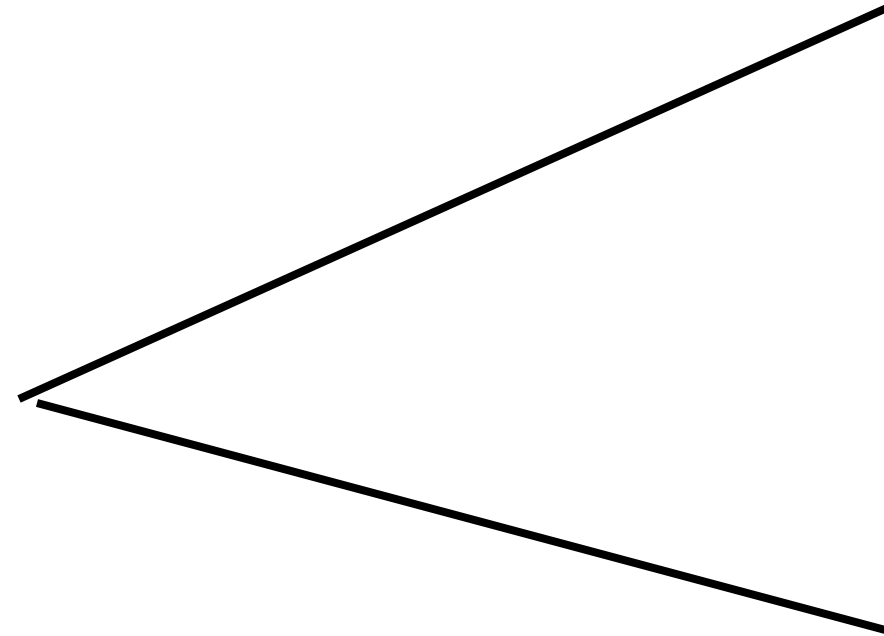
- Extraire automatiquement le contenu textuel de fichiers PDF.
- Résumer les documents pédagogiques pour faciliter la compréhension.
- Générer automatiquement des quiz interactifs à partir du contenu.
- Proposer une interface simple et intuitive pour les utilisateurs.



La cible



Utilisateur

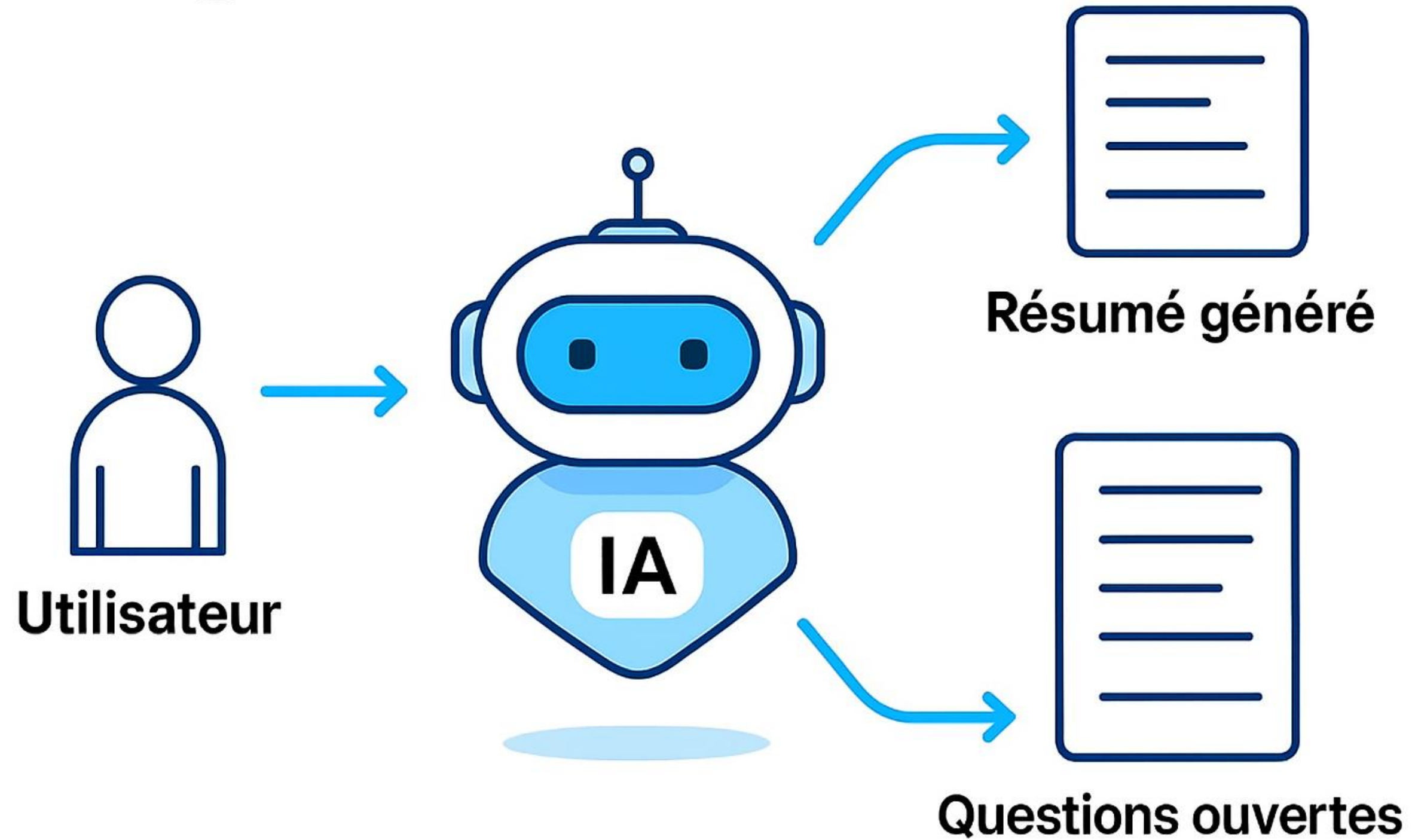


Chercheurs et
enseignants



Etudiants

Cas d'usage



Fonctionnalités principales



LECTURE ET EXTRACTION DE
TEXTES DEPUIS DES PDFS.



TRADUCTION AUTOMATIQUE
(FRANÇAIS ↔ ANGLAIS).



RÉSUMÉ GÉNÉRÉ MÊME SUR
FICHIERS VOLUMINEUX.



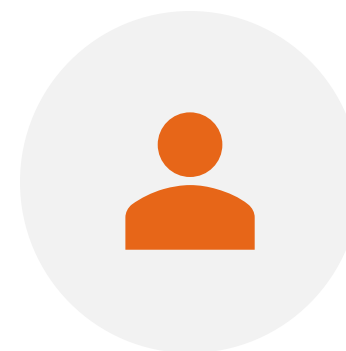
RÉSUMÉ AVEC STUDENT PAR
DÉFAUT,



TEACHER COMME FALLBACK
SI PROBLÈME.



GÉNÉRATION DE QUIZ.



GÉNÉRATION DE RÉSUMÉ
SPÉCIFIQUES AVEC UN
PROMPT UTILISATEUR

Transformez vos PDF en résumés intelligents

Résumé & quiz automatisés

Déposez votre fichier ici (PDF)

Choisir le fichier aucun fichier sélectionné

Posez une question (facultatif) :

Exemple : Quels sont les points clés abordés ?

Analyser

Aperçu du document PDF

Blockchain et développement

Helder SALVADOR, Développeur
Blockchain et Directeur Pédagogique
filière Blockchain ESGI

03/09/20XX

2



Questions ouvertes générées à partir du PDF

Q1. {'question': "l'année majeure a fait la majeure majeure majeure"}

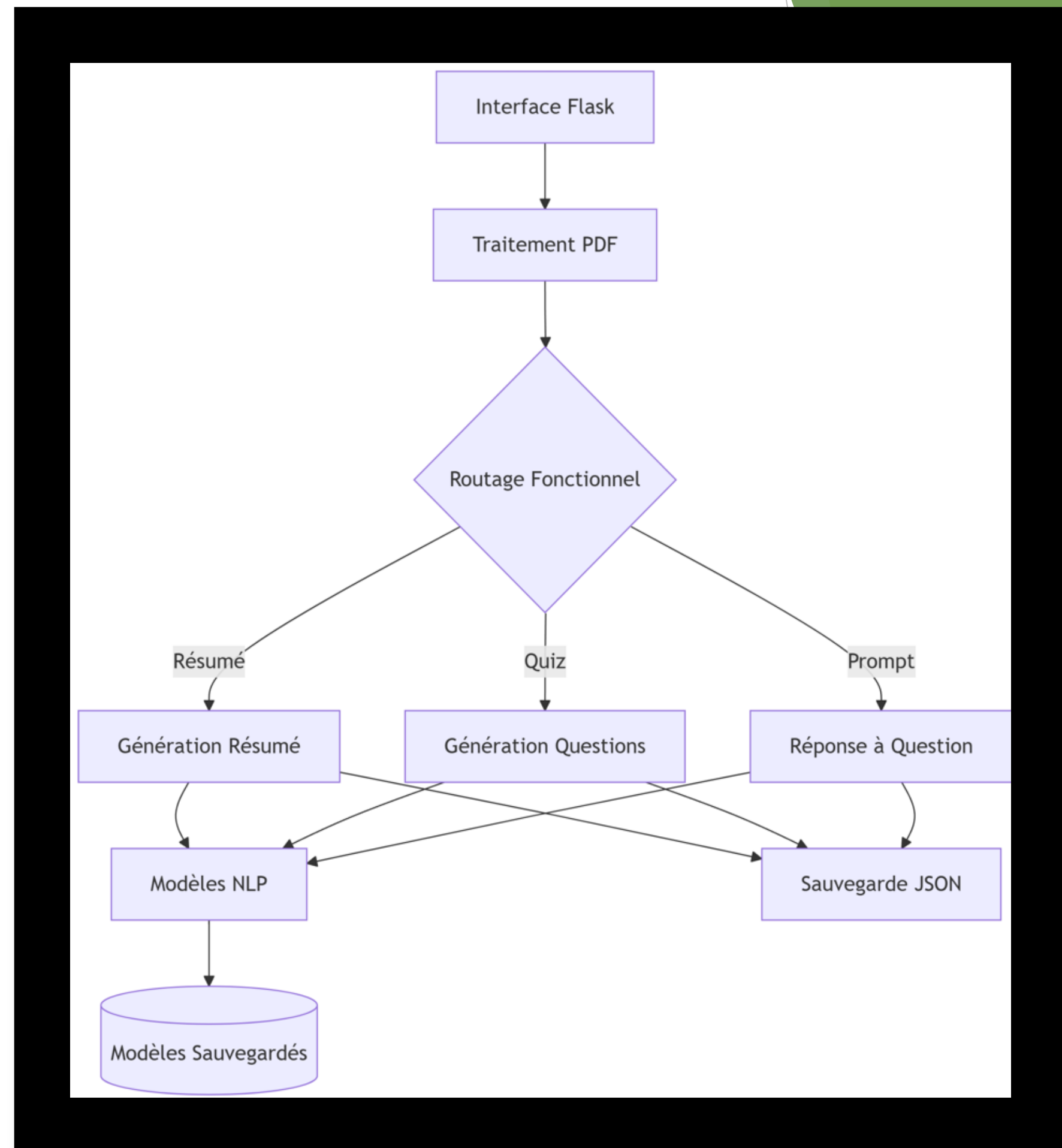
Votre réponse ici...

Q2. {'question': "qu'a-t-il commencé à utiliser en porcelaine ?"}

Votre réponse ici...

Q3. {'question': 'quel type de problème sensoriel les modèles de transfert ont-ils

Architecture global



Outils et plateformes technologiques

01

Développement IA/NLP:
Python, Hugging Face
Transformers

02

Utilisation de la lib
argotranslate pour la
traduction des pdf

03

Traitement texte
Avec
ByteLevelBPETokenize
r

04

Entraînement des
models avec Kaggle et
colab pro

05

Déploiement cloud +
Entraînement modèle
(Cloud ML): AWS (EC2) +
S3

06

Interface utilisateur :
Flask

Nos pipelines : Pipeline de génération de résumé

- ▶ 1. Extraction PDF
- ▶ Extraction et Nettoyage texte (Fitz)
- ▶ Détection langue (FR/EN)
- ▶ 2. Préparation: Traduction FR→EN et découpage en chunks intelligents
- ▶ 3. Génération : Modèle Étudiant : Résumés partiels
- ▶ Vérification cohérence :
 - ▶ Longueur min.
 - ▶ Similarité sémantique
- ▶ Fallback : Modèle Enseignant (BART)
- ▶ 4. Structuration : Regroupement thématique (K-Means)

```
def summarize_pdf(pdf_file, max_chunk_len=30, p=0.9, threshold=0.6):  
    """Résumé détaillé du contenu d'un PDF donné en entrée (bytes)."""  
    try:  
        print("[summarize_pdf] Début du processus...")  
        raw_text = processor.extract_text(pdf_file)  
        print(f"[summarize_pdf] Extrait brut (1000 chars max): {raw_text[:1000]}")  
        print(f"[summarize_pdf] Longueur du texte extrait : {len(raw_text)} caractères.")  
  
        # Détection de la langue  
        lang_detected = detect(raw_text)  
        print(f"[INFO] Langue du texte détectée : {lang_detected}")  
  
        # Si français, traduction en anglais pour le modèle  
        if lang_detected == "fr":  
            print("[INFO] Texte en français, traduction en anglais pour le modèle...")  
            text_for_model = translate_fr_to_en(raw_text)  
        else:  
            text_for_model = raw_text  
            print(f"[INFO] Texte en {lang_detected}, pas de traduction nécessaire pour le modèle.")  
  
        # Découpe du texte  
        max_chunks_for_test = 5  
        chunks = processor.chunk_and_tokenize(text_for_model, max_chunk_len)[:max_chunks_for_test]  
        summaries = []  
        for idx, chunk in enumerate(chunks):  
            print(f"[summarize_pdf] Traitement du chunk {idx + 1}/{len(chunks)}...")  
            src_ids = torch.tensor(data=[chunk], device=device)  
            src_mask = (src_ids != pad_id).long()  
            summary_en = nucleus_sampling_decode(model, src_ids, src_mask, tokenizer, p=p)  
            summaries.append(clean_and_format_summary(summary_en))
```

Nos pipelines : Pipeline de génération de quizz

- ▶ Extraction Texte avec PyMuPDF
- ▶ Préparation Contenu : Découpage en chunks de 200 mots
- ▶ Sélection max. 5 passages clés
- ▶ 3. Génération Questions
- ▶ Modèle : FLAN-T5-small
- ▶ Paramètres :
 - ▶ Sampling créatif (top-p=0.95)
 - ▶ Température=0.9
 - ▶ Longueur max=64 tokens
- ▶ Détection langue automatique
- ▶ Traduction EN→FR si nécessaire
- ▶ Utilisation Helsinki-NLP
- ▶ 5. Sortie : Liste de questions ouvertes

```
def generer_question(texte, model, vocab, device, max_len=30):
    model.eval()
    src = torch.tensor(vocab.encode(texte)).unsqueeze(0).to(device)
    encoder_outputs = model.encoder(src)
    hidden = encoder_outputs[:, -1, :].unsqueeze(0)
    input = torch.tensor([[vocab.word2idx["<sos>"]]]).to(device)
    output_indices = []

    for _ in range(max_len):
        output, hidden = model.decoder(input, hidden, encoder_outputs)
        top1 = output.argmax(1)
        output_indices.append(top1.item())
        if top1.item() == vocab.word2idx["<eos>"]:
            break
        input = top1.unsqueeze(1)

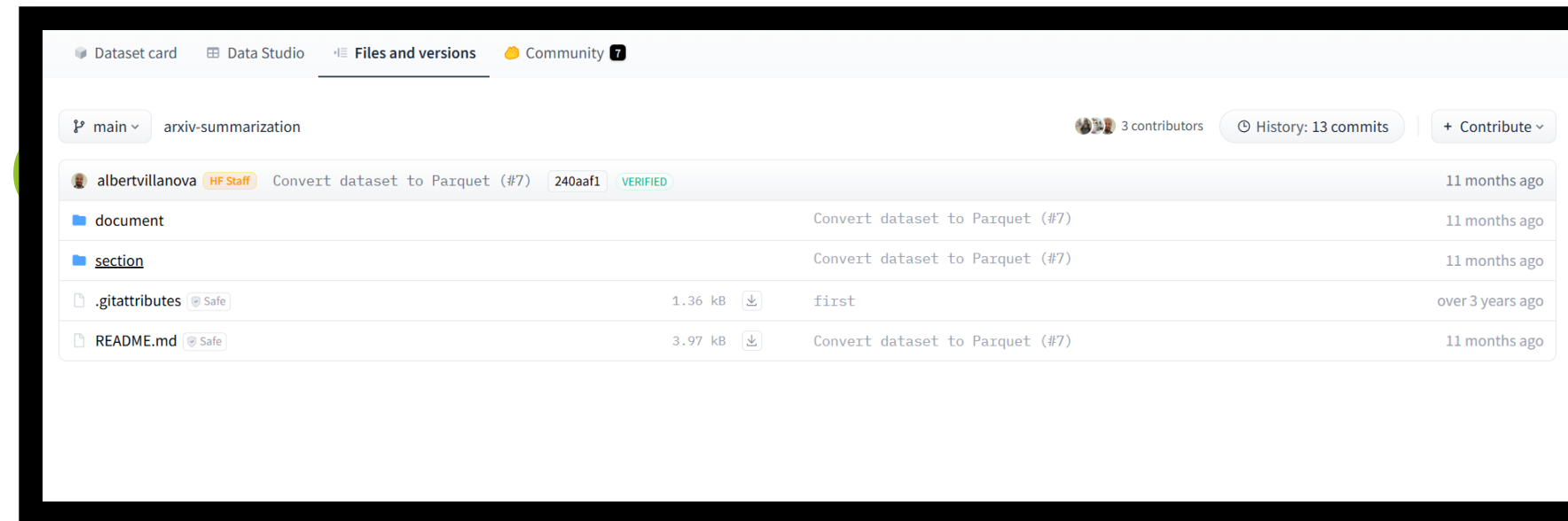
    return vocab.decode(output_indices)

def decouper_en_phrases(text):

    sentences = re.split(r'(?<=[.!?])\s+', text.strip())
    return [s for s in sentences if len(s) > 30]
```

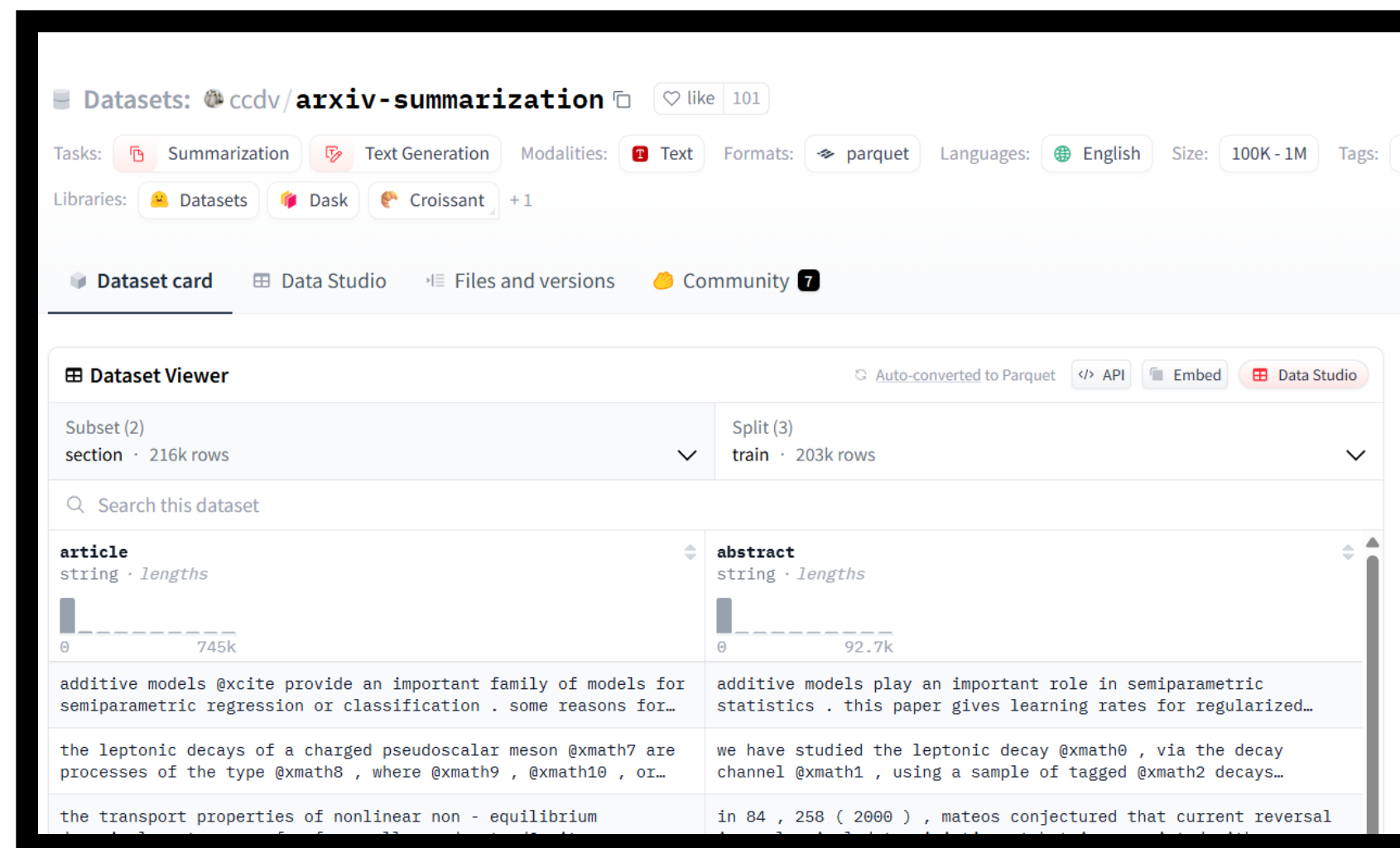
Dataset d'entraînement

► Dataset résumé



This screenshot shows the 'Files and versions' tab of the 'arxiv-summarization' dataset on Hugging Face. The page lists the commit history, showing a recent commit by 'albertvillanova' that converted the dataset to Parquet format. The file structure includes a 'document' directory, a 'section' directory, and two files: '.gitattributes' and 'README.md'.

Commit	Author	Message	SHA	Status	Time
main	albertvillanova	Convert dataset to Parquet (#7)	240aaf1	VERIFIED	11 months ago
document		Convert dataset to Parquet (#7)			11 months ago
section		Convert dataset to Parquet (#7)			11 months ago
.gitattributes		first			over 3 years ago
README.md		Convert dataset to Parquet (#7)			11 months ago



This screenshot shows the 'Dataset Viewer' for the 'arxiv-summarization' dataset. It displays the dataset's metadata, including the number of rows in the 'section' subset (216k) and the 'train' split (203k). The viewer shows two columns: 'article' and 'abstract', both of type 'string'. The 'article' column has a length of 745k, and the 'abstract' column has a length of 92.7k. The viewer also displays sample text from both columns.

Subset (2)	Split (3)
section · 216k rows	train · 203k rows

article	abstract
string · lengths	string · lengths
0 745k	0 92.7k
additive models @xcite provide an important family of models for semiparametric regression or classification . some reasons for...	additive models play an important role in semiparametric statistics . this paper gives learning rates for regularized...
the leptonic decays of a charged pseudoscalar meson @xmath7 are processes of the type @xmath8 , where @xmath9 , @xmath10 , or...	we have studied the leptonic decay @xmath0 , via the decay channel @xmath1 , using a sample of tagged @xmath2 decays...
the transport properties of nonlinear non - equilibrium	in 84 , 258 (2000) , mateos conjectured that current reversal

Prétraitement

- Dataset résumé
- Tokenisation avec padding et nettoyage des symbols non textes

```
# Load tokenizer with academic tokens
tokenizer = ByteLevelBPETokenizer(
    "/kaggle/input/tokenizer/vocab_cleaned.json",
    "/kaggle/input/tokenizer/merges_clean.txt"
)
tokenizer.add_special_tokens(["[CITE]", "[MATH]", "[EQUATION]"])

# Verify special tokens
print("Special tokens:")
print(f"<pad>: {tokenizer.token_to_id('<pad>')}")
print(f"<s>: {tokenizer.token_to_id('<s>')}")
print(f"</s>: {tokenizer.token_to_id('</s>')}")
print(f"[CITE]: {tokenizer.token_to_id('[CITE]')}")
print(f"[MATH]: {tokenizer.token_to_id('[MATH]')}")

# Create datasets
train_dataset = AcademicSummaryDataset(
    "/kaggle/input/corpus-train-val/corpus_train_clean.txt",
    tokenizer,
    max_src_len=1024,
    max_tgt_len=256
)
```

```
sample = train_dataset[0]
print("\nProcessed sample:")
print("Source:", sample["src_text"][:200] + "...")
print("Target:", sample["tgt_text"])
print(f"Source IDs shape: {sample['src_ids'].shape}")
print(f"Target IDs shape: {sample['tgt_ids'].shape}")
```

Special tokens:

```
<pad>: 1
<s>: 0
</s>: 2
[CITE]: 40000
[MATH]: 40001
```

Processed sample:

Source: the interest in anchoring phenomena and phenomena in confined nematic liquid crystals has largely been driven by their potential use in liquid crystal display devices . the twisted nematic liquid crystal display is a type of liquid crystal display in which the liquid crystal molecules are aligned in a plane parallel to the surface of the substrate . the twisted nematic liquid crystal display is a type of liquid crystal display in which the liquid crystal molecules are aligned in a plane parallel to the surface of the substrate . the twisted nematic liquid crystal display is a type of liquid crystal display in which the liquid crystal molecules are aligned in a plane parallel to the surface of the substrate .

Target: we study the phase behavior of a nematic liquid crystal confined between a flat substrate with a patterned substrate whose structure and local anchoring strength we vary . by first evaluating the free energy of the confined nematic liquid crystal . then we determine phase diagrams involving a homogeneous state in which the nematic director is almost uniform and a hybrid aligned nematic state in which the director varies through the cell . direct minimization of the free energy functional were performed to test the predictions of the effective free energy method . we find remarkably good agreement between the boundaries calculated from the two approaches . in addition the effective energy method allows one to determine energy barriers between two states in a bistable nematic device .

Source IDs shape: torch.Size([1024])

Target IDs shape: torch.Size([156])

+ Code

+ Markdown

Dataset quizz

8

Datasets:  vivekraina/Stanford_Question_Answering_Dataset   like 0

 Dataset card  Files and versions  Community

Dataset Viewer

 API  Embed  Data Studio

Split (2)
train 

► The dataset viewer is not available for this split.

Downloads last month 26

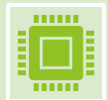


Architecture des modèles de résumés

12



Teacher model :



BART pré-entraîné (Facebook/bart-large-cnn), fine-tuné sur le dataset arXiv-summarization.



Student model :



modèle encodeur-décodeur basé sur le Transformer avec ByteLevelBPETokenizer.



Paramètres : 6 layers, 12 heads, 768-dim, dropout 0.2, gradient checkpointing activé

```
model = TransformerSummarizer(  
    vocab_size,  
    pad_id=pad_id,  
    d_model=768,  
    nhead=12,  
    num_layers=6,  
    dropout=0.2,  
    use_checkpointing=False  
)  
.to(device)  
model.load_state_dict(torch.load(checkpoint_path, map_location=device))  
model.eval()
```

```
print('F1 score : train={1en(dataset_small['train'])}, val={1en(dataset_sm  
  
# PREPARER MODELE  
model_name = "facebook/bart-large-cnn"  
tokenizer = BartTokenizer.from_pretrained(model_name)  
model = BartForConditionalGeneration.from_pretrained(model_name)  
data_collator = DataCollatorForSeq2Seq(tokenizer, model)  
  
# CALLBACK POUR S'ENREGISTRER
```

Défis rencontrés pendant l'apprentissage

Taille massive du dataset (~300 000 exemples).

Entraînement Modèle from scratch très coûteux en ressources.

Limites de temps/ressources sur Colab & Kaggle.

Besoin d'un pipeline robuste pour manipuler des fichiers volumineux.

Evolutions et solutions choisies

Passage d'un entraînement from scratch → knowledge distillation.

Introduction d'un teacher model pré-entraîné (BART fine-tuné).

Découpage du dataset en sections diversifiées (sampling avec seed).

Checkpoints et reprise automatique (environnement instable Colab/Kaggle).

Pipeline split + merge sur les PDFs pour gérer les longues séquences.

```
# === Entraînement avec distillation ===
for epoch in range(start_epoch, total_epochs):
    student_model.train()
    for batch_idx, batch in enumerate(train_loader):
        if epoch == start_epoch and batch_idx < start_batch_idx:
            continue

        src_ids = batch['src_ids'].to('cuda')
        src_mask = batch['src_mask'].to('cuda')
        tgt_ids = batch['tgt_ids'].to('cuda')
        tgt_mask = batch['tgt_mask'].to('cuda')
        decoder_input = tgt_ids[:, :-1]
        decoder_target = tgt_ids[:, 1:]
        decoder_mask = tgt_mask[:, :-1]

        with torch.no_grad():
            teacher_outputs = teacher_model(
                input_ids=src_ids,
                attention_mask=src_mask,
                decoder_input_ids=decoder_input
            )
            teacher_logits = teacher_outputs.logits

        with autocast():
            student_outputs = student_model(
                src_ids=src_ids,
                tgt_ids=decoder_input,
                src_mask=src_mask,
                tgt_mask=decoder_mask
            )
```

```
[8]: from torch.utils.data import DataLoader
      from datasets import DatasetDict

      # === Réduction diversifiée ===
      session_size = 10000
      train_dataset_reduced = train_dataset.shuffle(seed=42).select(range(session_size))

      # === Model ===
      vocab_size = tokenizer.get_vocab_size()
      pad_id = tokenizer.token_to_id("<pad>")

      model = TransformerSummarizer(
```

Apprentissage continue du model custom avec la distillation

Apprentissage avec Full Knowledge distillation :

Input → Teacher → Softmax(logits) → KL Divergence → Student → Optimizer

Le student apprend à produire la même distribution de sortie que le teacher, pas juste à prédire le bon mot.

```
Epoch 14 terminé | Loss moyenne: 1.3577
Epoch 15: 100%|██████████| 625/625 [08:23<00:00, 1.24batch/s, avg_loss=1.38, loss=1.26]
✓ Epoch 15 terminé | Loss moyenne: 1.3773
Epoch 16: 100%|██████████| 625/625 [08:22<00:00, 1.24batch/s, avg_loss=1.35, loss=1.39]
✓ Epoch 16 terminé | Loss moyenne: 1.3524
Epoch 17: 100%|██████████| 625/625 [08:23<00:00, 1.24batch/s, avg_loss=1.33, loss=1.42]
✓ Epoch 17 terminé | Loss moyenne: 1.3324
Epoch 18: 100%|██████████| 625/625 [08:22<00:00, 1.24batch/s, avg_loss=1.31, loss=1.29]
✓ Epoch 18 terminé | Loss moyenne: 1.3091
Epoch 19: 100%|██████████| 625/625 [08:23<00:00, 1.24batch/s, avg_loss=1.29, loss=1.14]
✓ Epoch 19 terminé | Loss moyenne: 1.2882
Epoch 20: 100%|██████████| 625/625 [08:23<00:00, 1.24batch/s, avg_loss=1.27, loss=1.58]
✓ Epoch 20 terminé | Loss moyenne: 1.2680
🌀 Modèle final sauvegardé : /kaggle/working/checkpoints/final_student_model_session_kaggle_last.pth
```

+ Code

+ Markdown

Pourquoi on a fait ça

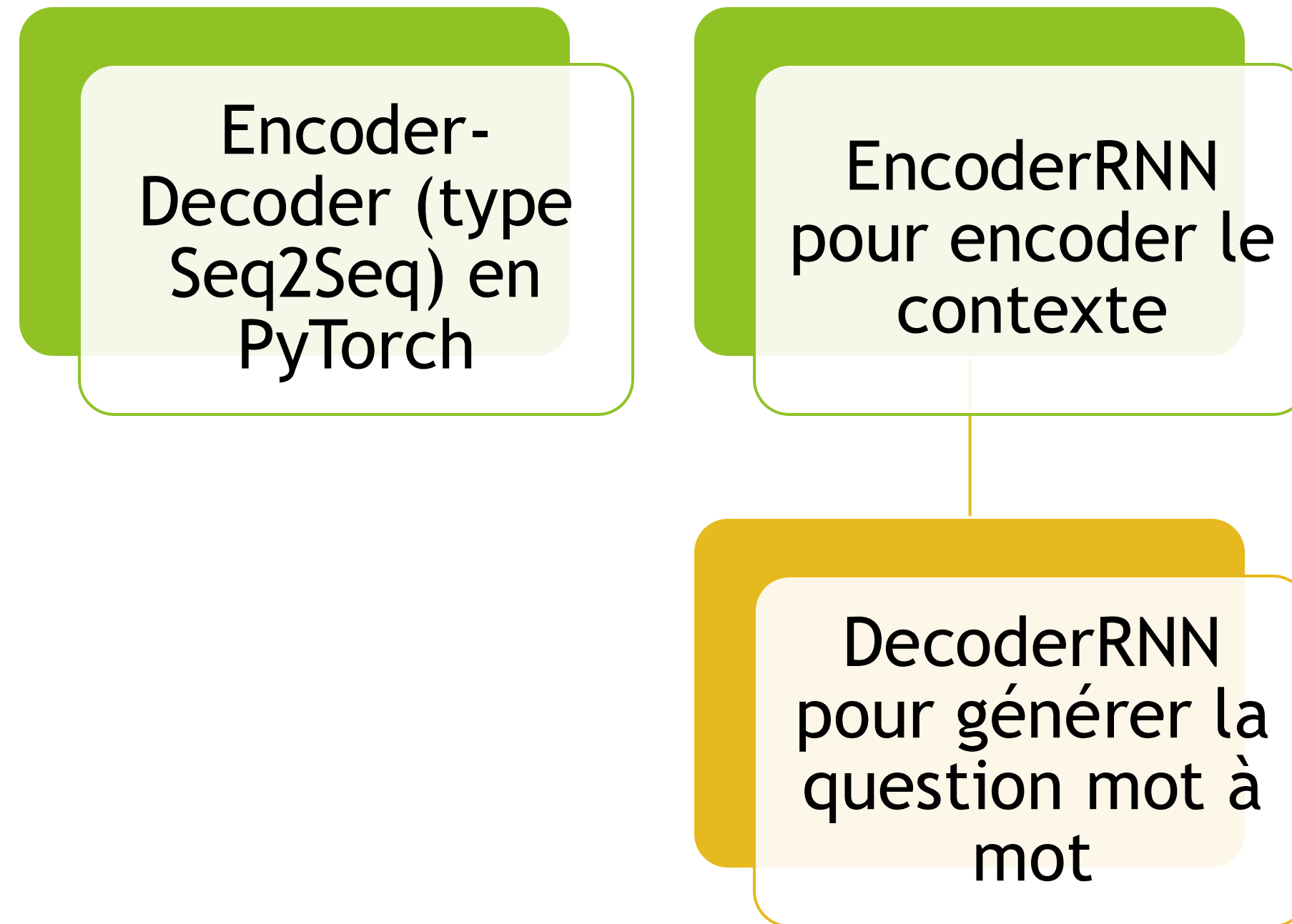
- ✨ Réduit la **taille du modèle final** (student léger).
- ✨ Garde une **qualité proche** du modèle teacher.
- ✨ Peut fonctionner même **sans cibles annotées** (juste avec les outputs teacher).

Comparaison avec entraînement normal sur le Dataset

- ▶ Jusqu'à 60 epochs pour un loss toujours élevé de 3.3
- ▶ Temps d'entraînement plus 6hr
- ▶ Plus de ressources consommé sur collab

```
==== Epoch 56/60 ====  
Epoch 56 | Batch 537/537 | Loss: 3.3462  
✓ Epoch 56 terminé | Loss moyen: 3.4108 | Perplexity: 30.29  
  
==== Epoch 57/60 ====  
Epoch 57 | Batch 537/537 | Loss: 3.5516  
✓ Epoch 57 terminé | Loss moyen: 3.4058 | Perplexity: 30.14  
  
==== Epoch 58/60 ====  
Epoch 58 | Batch 537/537 | Loss: 3.3645  
✓ Epoch 58 terminé | Loss moyen: 3.3996 | Perplexity: 29.95  
  
==== Epoch 59/60 ====  
Epoch 59 | Batch 537/537 | Loss: 3.3779  
✓ Epoch 59 terminé | Loss moyen: 3.3969 | Perplexity: 29.87  
  
==== Epoch 60/60 ====  
Epoch 60 | Batch 537/537 | Loss: 3.4746  
✓ Epoch 60 terminé | Loss moyen: 3.3956 | Perplexity: 29.83  
⌘ Modèle final sauvegardé : /content/drive/MyDrive/student_chec
```


Architecture du Modèle seq2seq (Question)



Modèle pour les questions



FLAN-T5 (Google)



Objectif : créer des questions ouvertes à partir du texte du PDF



Méthode : prompt + découpage du texte



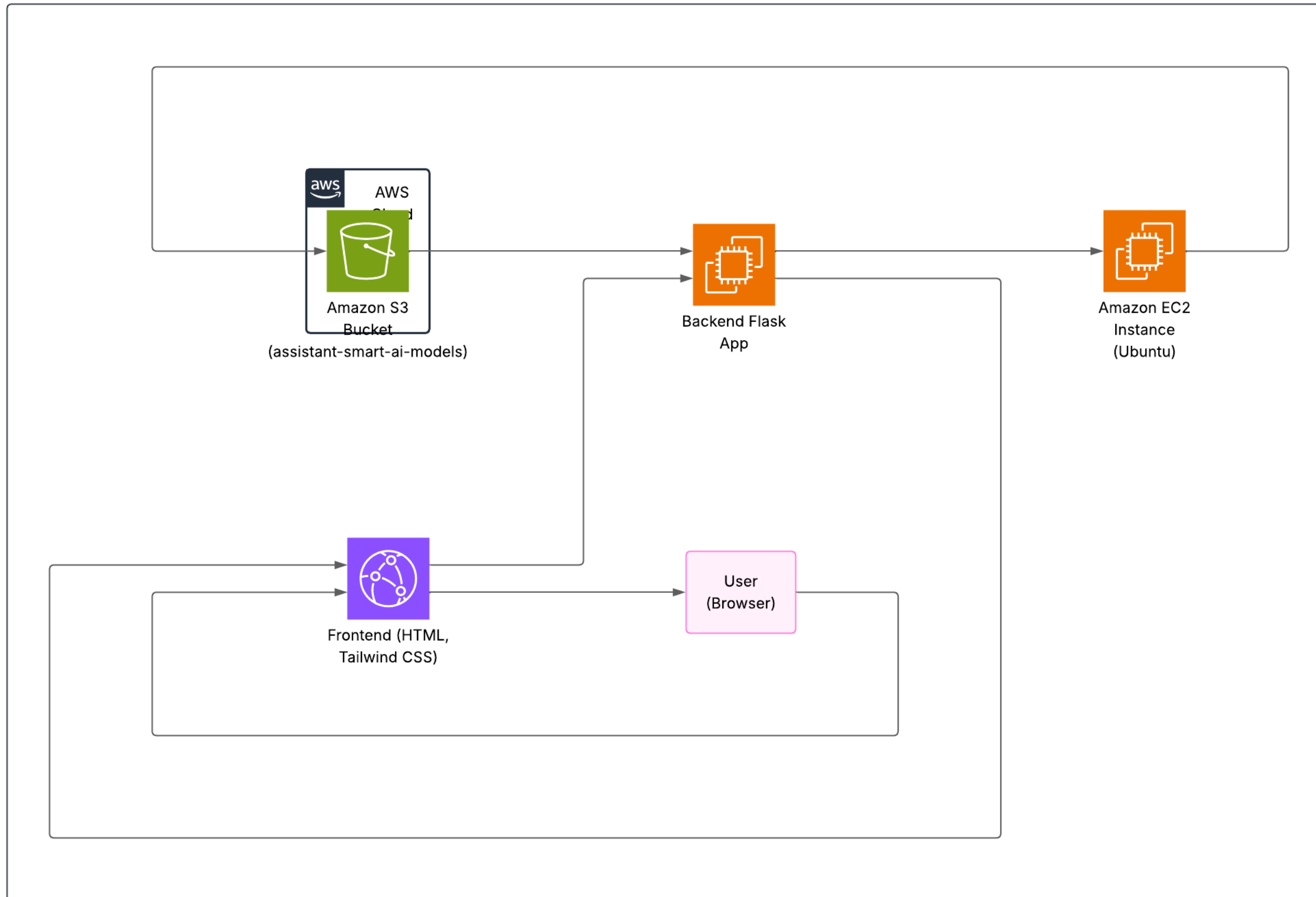
Traduction automatique en français



5 questions générées par document

Architecture cloud

AWS Cloud Architecture for Flask-based AI Web Application



Architecture cloud

Élément	Description
EC2 (Instance Ubuntu)	Héberge l'application Flask (frontend HTML + backend Python).
Flask	Micro-framework qui gère les routes, le rendu HTML, et la logique métier.
S3 (Amazon S3)	Stocke les modèles IA (poids .pt, .json...) et les PDF uploadés.
Modèles IA (locaux)	Chargés depuis S3 au premier lancement (download_from_s3).
HTML (Jinja2 + Tailwind)	Affiche les formulaires, les résultats, les quizz.

Perspective futures

Amélioration des post-processings.

Augmentation du corpus multi-langues

Améliorer la généralisation de nos models

Génération de QCM et flashcards à partir des résumés.

**Merci pour votre
attention !**

